ANALYSIS OF ALGORITHMS

Master Guidance Document

Dynamic Programming • Memoization • Greedy Algorithms • Graph Algorithms • NP-Completeness • Approximation Algorithms

#	Topic	Chapter
1	Dynamic Programming (Knapsack, Coin Change, Rod Cutting, LCS, Matrix Chain, Optimal BST…)	Ch 15
2	Memoization Overview (Fibonacci)	—
3	Greedy Algorithms (Activity Selection, Huffman, TSP, Knapsack)	Ch 16
4	Elementary Graph Algorithms (BFS, DFS)	Ch 22
5	NP-Completeness Overview	Ch 34
6	Approximation Algorithms Overview	Ch 35

CHAPTER 1: DYNAMIC PROGRAMMING

CLRS Chapter 15

Dynamic Programming (DP) solves complex problems by breaking them into overlapping subproblems, storing solutions to avoid recomputation. Two properties must hold: **Optimal Substructure** (optimal solution contains optimal solutions to subproblems) and **Overlapping Subproblems** (subproblems recur many times).

Two approaches:

- **Top-Down (Memoization):** Recursive + cache results
- **Bottom-Up (Tabulation):** Iterative, fill table from smallest subproblems

1.1 0/1 Knapsack Problem

Given n items with weights w[i] and values v[i], and a knapsack of capacity W, find the maximum value achievable without exceeding W. Each item is either taken (1) or not (0).

Recurrence:

```
dp[i][w] = max value using first i items with capacity w


dp[i][w] = dp[i-1][w] if w[i] > w

dp[i][w] = max(dp[i-1][w], otherwise

v[i] + dp[i-1][w - w[i]])


Base case: dp[0][w] = 0 for all w
```

Pseudocode:

```
KNAPSACK(w[], v[], n, W):

for i = 0 to n: dp[i][0] = 0

for j = 0 to W: dp[0][j] = 0

for i = 1 to n:

for j = 1 to W:

if w[i] > j:

dp[i][j] = dp[i-1][j]

else:

dp[i][j] = max(dp[i-1][j], v[i] + dp[i-1][j - w[i]])

return dp[n][W]
```

C++ Code:

```
#include <bits/stdc++.h>

using namespace std;


int knapsack(vector<int>& w, vector<int>& v, int n, int W) {

vector<vector<int>> dp(n+1, vector<int>(W+1, 0));

for (int i = 1; i <= n; i++) {

for (int j = 0; j <= W; j++) {

dp[i][j] = dp[i-1][j]; // don't take item i

if (w[i-1] <= j) // take item i

dp[i][j] = max(dp[i][j], v[i-1] + dp[i-1][j - w[i-1]]);

}

}

return dp[n][W];

}
```

Example: n=3, W=5, weights=[2,3,4], values=[3,4,5]

dp table → Answer = 7 (items 1+2, weight=2+3=5, value=3+4=7)

Metric	Value
Time Complexity	O(nW)
Space Complexity	O(nW) → O(W) with 1D
Optimal Substructure	Yes
Overlapping Subproblems	Yes

1.2 Coin Change

Given coin denominations and amount S, find minimum number of coins to make amount S.

Recurrence:

```
dp[0] = 0

dp[i] = min(dp[i - coin[j]] + 1) for all coins j where coin[j] <= i

dp[i] = INF if no solution exists
```

Pseudocode:

```
COIN_CHANGE(coins[], n, S):
```

```
dp[0..S] = INF

dp[0] = 0

for i = 1 to S:

for each coin c in coins[]:

if c <= i and dp[i-c] + 1 < dp[i]:

dp[i] = dp[i-c] + 1

return dp[S] if dp[S] != INF else -1
```

C++ Code:

```
int coinChange(vector<int>& coins, int amount) {

vector<int> dp(amount+1, INT_MAX);

dp[0] = 0;

for (int i = 1; i <= amount; i++) {

for (int c : coins) {

if (c <= i && dp[i-c] != INT_MAX)

dp[i] = min(dp[i], dp[i-c] + 1);

}

}

return dp[amount] == INT_MAX ? -1 : dp[amount];

}
```

Example: coins=[1,5,6,9], amount=11 → 2 coins (5+6)

Time	Space
O(S * n)	O(S)

1.3 Rod Cutting

Given a rod of length n and price table p[i], find max revenue from cutting the rod.

```
Recurrence: r[n] = max(p[i] + r[n-i]) for i = 1 to n

r[0] = 0
```

C++ Code:

```
int rodCutting(vector<int>& p, int n) {
```

```
vector<int> r(n+1, 0);

for (int j = 1; j <= n; j++) {

int q = INT_MIN;

for (int i = 1; i <= j; i++)

q = max(q, p[i-1] + r[j-i]);

r[j] = q;

}

return r[n];

}
```

Example: p=[1,5,8,9,10,17,17,20], n=4 → r[4]=10 (cut into 2+2)

Time	Space
$O(n^2)$	$O(n)$

1.4 Maximum Subarray Sum (Kadane's Algorithm)

Find contiguous subarray with largest sum in array A[1..n].

```
dp[i] = max subarray sum ending at index i

dp[i] = max(A[i], dp[i-1] + A[i])

Answer = max(dp[i]) for all i
```

C++ Code:

```
int maxSubarray(vector<int>& A) {

int cur = A[0], best = A[0];

for (int i = 1; i < A.size(); i++) {

cur = max(A[i], cur + A[i]);

best = max(best, cur);

}

return best;

}
```

Example: A=[-2,1,-3,4,-1,2,1,-5,4] → 6 (subarray [4,-1,2,1])

Time	Space

O(n)	O(1)

1.5 Longest Common Subsequence (LCS)

Find longest subsequence present in both sequences X[1..m] and Y[1..n]. A subsequence need not be contiguous.

```
dp[i][j] = LCS length of X[1..i] and Y[1..j]


dp[i][j] = dp[i-1][j-1] + 1 if X[i] == Y[j]

dp[i][j] = max(dp[i-1][j], dp[i][j-1]) otherwise

dp[0][j] = dp[i][0] = 0
```

C++ Code:

```
int lcs(string X, string Y) {

int m = X.size(), n = Y.size();

vector<vector<int>> dp(m+1, vector<int>(n+1, 0));

for (int i = 1; i <= m; i++)

for (int j = 1; j <= n; j++)

if (X[i-1] == Y[j-1]) dp[i][j] = dp[i-1][j-1] + 1;

else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);

return dp[m][n];

}
```

Example: X="ABCBDAB", Y="BDCABA" → LCS=4 ("BCBA")

Time	Space
O(mn)	O(mn)

1.6 Chain Matrix Multiplication

Find optimal parenthesization of matrix chain A1·A2·…·An to minimize scalar multiplications. Matrix Ai has dimension p[i-1] x p[i].

```
m[i][j] = min cost to multiply matrices i through j


m[i][i] = 0

m[i][j] = min over k from i to j-1 of:
```

```
m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j]
```

C++ Code:

```
int matrixChain(vector<int>& p, int n) {

// n matrices, p has n+1 dimensions

vector<vector<int>> m(n+1, vector<int>(n+1, 0));

for (int len = 2; len <= n; len++) {

for (int i = 1; i <= n-len+1; i++) {

int j = i + len - 1;

m[i][j] = INT_MAX;

for (int k = i; k < j; k++) {

int cost = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];

m[i][j] = min(m[i][j], cost);

}

}

}

return m[1][n];

}
```

Example: p=[30,35,15,5,10,20,25], n=6 → 15125 multiplications

Time	Space
$O(n^3)$	$O(n^2)$

1.7 Convex Polygon Triangulation

Minimum-weight triangulation of a convex polygon with n vertices. Weight of triangle (i,j,k) is sum of its side lengths. Similar structure to matrix chain multiplication.

```
dp[i][j] = min cost triangulation of polygon from vertex i to j


dp[i][i+1] = 0 (edge, no triangle)

dp[i][j] = min over k (i < k < j) of:

dp[i][k] + dp[k][j] + weight(i, k, j)
```

Time	Space

$O(n^3)$	$O(n^2)$

1.8 Optimal Binary Search Tree

Given sorted keys k1 < k2 < … < kn with search probabilities p[i] and dummy key probabilities q[i], build BST minimizing expected search cost.

```
e[i][j] = expected cost for subtree with keys i..j

w[i][j] = sum of probabilities p[i..j] + q[i-1..j]


e[i][i-1] = q[i-1] (empty subtree)

e[i][j] = min over r from i to j of:

e[i][r-1] + e[r+1][j] + w[i][j]
```

C++ Code (skeleton):

```
void optimalBST(vector<double>& p, vector<double>& q, int n,

vector<vector<double>>& e, vector<vector<int>>& root) {

vector<vector<double>> w(n+2, vector<double>(n+1));

e.assign(n+2, vector<double>(n+1));

root.assign(n+1, vector<int>(n+1));

for (int i = 1; i <= n+1; i++) { e[i][i-1] = q[i-1]; w[i][i-1] = q[i-1]; }

for (int len = 1; len <= n; len++) {

for (int i = 1; i <= n-len+1; i++) {

int j = i + len - 1;

e[i][j] = 1e18;

w[i][j] = w[i][j-1] + p[j] + q[j];

for (int r = i; r <= j; r++) {

double t = e[i][r-1] + e[r+1][j] + w[i][j];

if (t < e[i][j]) { e[i][j] = t; root[i][j] = r; }

}

}

}

}
```

Time	Space
$O(n^3)$	$O(n^2)$

CHAPTER 2: MEMOIZATION (Overview)

Top-Down Dynamic Programming

Memoization stores results of expensive function calls and returns cached results when same inputs reoccur. It's the top-down counterpart to bottom-up tabulation.

Key Properties:

- Recursive structure preserved (easier to write)
- Cache lookup before each recursive call
- Only computes necessary subproblems (lazy evaluation)

2.1 Fibonacci — Classic Memoization Example

```
Naive Recursive: F(n) = F(n-1) + F(n-2), F(0)=0, F(1)=1

Time: O(2^n) — exponential due to repeated subproblems


With Memoization:

memo[] = {-1, -1, ..., -1}

F(n): if memo[n] != -1: return memo[n]

memo[n] = F(n-1) + F(n-2)

return memo[n]

Time: O(n) — each subproblem solved once
```

C++ Code:

```
#include <bits/stdc++.h>

using namespace std;

map<int,long long> memo;


long long fib(int n) {

if (n <= 1) return n;

if (memo.count(n)) return memo[n];

return memo[n] = fib(n-1) + fib(n-2);

}
```

Memoization vs Tabulation:

Aspect	Memoization (Top-Down)	Tabulation (Bottom-Up)

Direction	Recursion + cache	Iteration
Subproblems	Only needed ones	All subproblems
Code style	Closer to math recurrence	Explicit loops
Space	Call stack overhead	No recursion overhead
When to use	Sparse subproblems	Dense subproblems

CHAPTER 3: GREEDY ALGORITHMS

CLRS Chapter 16

Greedy algorithms make locally optimal choices at each step, hoping to reach a global optimum. Unlike DP, greedy does NOT revisit past decisions. Works when the problem has the **Greedy Choice Property**: locally optimal = globally optimal.

Greedy vs DP:

- DP: considers ALL subproblem solutions before deciding
- Greedy: makes one choice, never reconsiders
- No correctness guarantee in greedy — must PROVE it works for each problem

3.1 Activity Selection Problem

Select maximum number of mutually compatible activities. Activity i runs from s[i] to f[i]. Two activities are compatible if intervals don't overlap.

Greedy Strategy: Always pick activity with earliest finish time.

```
ACTIVITY_SELECTION(s[], f[], n):

Sort activities by finish time f[]

selected = {a1}

k = 1 // last selected

for m = 2 to n:

if s[m] >= f[k]: // compatible

selected.add(am)

k = m

return selected
```

C++ Code:

```
vector<int> activitySelection(vector<int>& s, vector<int>& f) {

int n = s.size();

vector<int> idx(n); iota(idx.begin(),idx.end(),0);

sort(idx.begin(),idx.end(),[&](int a,int b){return f[a]<f[b];});

vector<int> res = {idx[0]};

int last = idx[0];
```

```
for (int i = 1; i < n; i++) {

if (s[idx[i]] >= f[last]) {

res.push_back(idx[i]);

last = idx[i];

}

}

return res;

}
```

Example: s=[1,3,0,5,8,5], f=[2,4,6,7,9,9] → select activities {0,1,3,4}

Time	Space
O(n log n)	O(n)

3.2 Fractional Knapsack (Greedy)

Unlike 0/1 Knapsack, fractions of items are allowed. Greedy works here: sort by value/weight ratio, take greedily.

```
FRACTIONAL_KNAPSACK(w[], v[], n, W):

Compute ratio[i] = v[i] / w[i] for all i

Sort items by ratio descending

total_value = 0, remaining = W

for each item i (in sorted order):

take = min(w[i], remaining)

total_value += take * ratio[i]

remaining -= take

if remaining == 0: break

return total_value
```

C++ Code:

```
double fractionalKnapsack(vector<int>& w, vector<int>& v, int W) {

int n = w.size();

vector<int> idx(n); iota(idx.begin(),idx.end(),0);
```

```
sort(idx.begin(),idx.end(),[&](int a,int b){

return (double)v[a]/w[a] > (double)v[b]/w[b];});

double total = 0; int rem = W;

for (int i : idx) {

if (rem <= 0) break;

int take = min(w[i], rem);

total += (double)take * v[i] / w[i];

rem -= take;

}

return total;

}
```

NOTE: 0/1 Knapsack CANNOT be solved optimally with greedy. Use DP.

Time	Space
O(n log n)	O(n)

3.3 Huffman Codes

Greedy algorithm for optimal prefix-free binary encoding. Characters with higher frequency get shorter codes. Build using a min-priority queue.

```
HUFFMAN(C): // C = set of characters with freq

n = |C|

Q = min-priority-queue(C) // keyed on freq

for i = 1 to n-1:

z = new node

z.left = x = EXTRACT-MIN(Q)

z.right = y = EXTRACT-MIN(Q)

z.freq = x.freq + y.freq

INSERT(Q, z)

return EXTRACT-MIN(Q) // root
```

C++ Code:

```
struct Node {
```

```
char ch; int freq;

Node *left, *right;

Node(char c, int f): ch(c),freq(f),left(nullptr),right(nullptr){}

};

struct Cmp { bool operator()(Node* a, Node* b){return a->freq > b->freq;} };


Node* buildHuffman(map<char,int>& freq) {

priority_queue<Node*,vector<Node*>,Cmp> pq;

for (auto& [c,f] : freq) pq.push(new Node(c,f));

while (pq.size() > 1) {

Node *x = pq.top(); pq.pop();

Node *y = pq.top(); pq.pop();

Node *z = new Node('\0', x->freq + y->freq);

z->left = x; z->right = y;

pq.push(z);

}

return pq.top();

}
```

Example: freq={a:45,b:13,c:12,d:16,e:9,f:5} → a=0 (1 bit), b=101 (3 bits), etc. Total cost = 224 bits (optimal).

Time	Space
O(n log n)	O(n)

3.4 Travelling Salesman Problem (Greedy Heuristic)

NP-Hard problem. Greedy nearest-neighbor heuristic: start from any city, repeatedly visit the nearest unvisited city. NOT guaranteed optimal.

```
GREEDY_TSP(cities[], n):

visited = {0}, tour = [0], current = 0

while |visited| < n:

nearest = argmin dist(current, j) for j not in visited

tour.append(nearest)
```

```
    visited.add(nearest)

    current = nearest

    tour.append(0) // return to start

    return tour
```

No correctness guarantee in greedy — can be far from optimal. See Ch 35 for approximation.

Time	Space
$O(n^2)$	$O(n)$

CHAPTER 4: ELEMENTARY GRAPH ALGORITHMS

CLRS Chapter 22

4.1 Representations of Graphs (22.1)

Two standard representations: **Adjacency Matrix** and **Adjacency List**.

Property	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	O(V + E)
Edge lookup	O(1)	O(degree(u))
Add edge	O(1)	O(1) amortized
Iterate neighbors	O(V)	O(degree(u))
Best for	Dense graphs	Sparse graphs

C++ Adjacency List:

```
int V = 5;

vector<vector<int>> adj(V); // adjacency list

// Add undirected edge u-v:

adj[u].push_back(v);

adj[v].push_back(u);


// Adjacency Matrix:

vector<vector<int>> mat(V, vector<int>(V, 0));

mat[u][v] = mat[v][u] = 1; // undirected
```

4.2 Breadth-First Search — BFS (22.2)

Explores graph level by level from source s. Computes shortest-path distances (in unweighted graphs). Uses a queue. Colors: WHITE (undiscovered), GRAY (in queue), BLACK (done).

```
BFS(G, s):

for each u in V - {s}:

u.color = WHITE, u.d = INF, u.pi = NIL

s.color = GRAY, s.d = 0, s.pi = NIL

Q = empty queue
```

```
ENQUEUE(Q, s)

while Q not empty:

u = DEQUEUE(Q)

for each v in Adj[u]:

if v.color == WHITE:

v.color = GRAY

v.d = u.d + 1

v.pi = u

ENQUEUE(Q, v)

u.color = BLACK
```

C++ Code:

```
void bfs(vector<vector<int>>& adj, int src, int V) {

vector<int> dist(V, -1), parent(V, -1);

queue<int> q;

dist[src] = 0;

q.push(src);

while (!q.empty()) {

int u = q.front(); q.pop();

for (int v : adj[u]) {

if (dist[v] == -1) {

dist[v] = dist[u] + 1;

parent[v] = u;

q.push(v);

}

}

}

}
```

BFS Tree: BFS produces a BFS-tree. dist[v] = shortest path from s to v (unweighted).

Time	Space	Shortest Path
O(V+E)	O(V)	Yes (unweighted)

4.3 Depth-First Search — DFS (22.3)

Explores as deep as possible before backtracking. Records discovery time d[u] and finish time f[u]. Produces DFS forest. Used for: topological sort, SCC, cycle detection.

```
DFS(G):
for each u in V: u.color=WHITE, u.pi=NIL
time = 0
for each u in V:
if u.color == WHITE: DFS-VISIT(G, u)

DFS-VISIT(G, u):
time = time + 1
u.d = time // discovery time
u.color = GRAY
for each v in Adj[u]:
if v.color == WHITE:
v.pi = u
DFS-VISIT(G, v)
u.color = BLACK
time = time + 1
u.f = time // finish time
```

C++ Code:

```
int timer = 0;
vector<int> disc, fin, parent;
vector<bool> visited;

void dfsVisit(vector<vector<int>>& adj, int u) {
visited[u] = true;
```

```
disc[u] = ++timer;

for (int v : adj[u]) {

if (!visited[v]) {

parent[v] = u;

dfsVisit(adj, v);

}

}

fin[u] = ++timer;

}


void dfs(vector<vector<int>>& adj, int V) {

disc.assign(V,0); fin.assign(V,0);

parent.assign(V,-1); visited.assign(V,false);

for (int u = 0; u < V; u++)

if (!visited[u]) dfsVisit(adj, u);

}
```

Edge Classification in DFS:

- **Tree edge:** v is WHITE when (u,v) explored
- **Back edge:** v is GRAY → creates cycle (in directed graph)
- **Forward edge:** v is BLACK, d[u] < d[v]
- **Cross edge:** v is BLACK, d[u] > d[v]

Time	Space	Applications
O(V+E)	O(V)	Topological Sort, SCC, Cycle Detection

CHAPTER 5: NP-COMPLETENESS (Overview)

CLRS Chapter 34

NP-Completeness theory classifies computational problems by their inherent difficulty. Key question: Does P = NP?

5.1 Problem Classes

Class	Definition	Example
P	Solvable in polynomial time	Sorting, BFS, shortest path
NP	Verifiable in polynomial time	Hamiltonian Cycle, SAT
NP-Complete	Hardest problems in NP; every NP problem reduces to it	3-SAT, TSP, Knapsack
NP-Hard	At least as hard as NP-Complete (may not be in NP)	Halting Problem

5.2 Key Comparisons

Shortest vs Longest Simple Path

Shortest path: EASY — O(V+E) with BFS/Dijkstra. Longest simple path: HARD — NP-Complete. No known polynomial algorithm.

Euler Tour vs Hamiltonian Cycle

Euler Tour (visits every EDGE once): EASY — O(E), exists iff every vertex has even degree. Hamiltonian Cycle (visits every VERTEX once): HARD — NP-Complete.

CNF-SAT vs 3-CNF-SAT

CNF-SAT: Is there an assignment satisfying a CNF formula? NP-Complete (Cook-Levin theorem). 3-CNF-SAT: Each clause has exactly 3 literals. Also NP-Complete.

Hamiltonian Cycle vs TSP

Ham. Cycle: Does graph have a Hamiltonian Cycle? NP-Complete. TSP: Find shortest Hamiltonian Cycle in weighted graph. NP-Complete. TSP is a generalization — Ham. Cycle reduces to TSP.

5.3 Reductions

Problem A reduces to B (A ≤p B) if: any instance of A can be solved using a polynomial-time algorithm for B. If B is easy, A is easy. If A is hard, B is hard.

```
To show problem B is NP-Complete:

1. Show B is in NP (poly-time verifier exists)

2. Show A ≤p B for some known NP-Complete problem A
```

```
(i.e., reduce A to B in polynomial time)


First NP-Complete problem: CIRCUIT-SAT (no prior problem to reduce from)

Cook-Levin Theorem: SAT is NP-Complete

Then: 3-SAT ≤p CLIQUE ≤p VERTEX-COVER ≤p HAM-CYCLE ≤p TSP
```

CHAPTER 6: APPROXIMATION ALGORITHMS (Overview)

CLRS Chapter 35

For NP-Hard problems, we often settle for near-optimal solutions in polynomial time. An approximation algorithm guarantees a solution within a factor ρ of the optimum.

6.1 Approximation Ratio

```
ρ(n) = max(C/C*, C*/C)


Where: C = cost of approximate solution

C* = cost of optimal solution

ρ = approximation ratio (ρ >= 1)


A ρ-approximation algorithm always produces solution within ρ * OPT
```

6.2 Vertex Cover Approximation (2-approximation)

Find minimum vertex cover (set of vertices touching all edges). Greedy matching gives 2-approximation.

```
APPROX-VERTEX-COVER(G):

C = empty set

E' = copy of all edges in G

while E' is not empty:

(u,v) = any edge in E'

C = C union {u, v}

remove all edges incident to u or v from E'

return C


Result: |C| <= 2 * |C*| (at most twice optimal)
```

Approximation Ratio	Time
2	O(V+E)

6.3 TSP Approximation (Metric TSP — 2-approximation)

When distances satisfy triangle inequality, use MST-based approximation.

```
APPROX-TSP-TOUR(G, c):

r = any vertex

T = MST of G rooted at r (using Prim/Kruskal)

H = preorder walk of T

return H (as Hamiltonian cycle)


Result: cost(H) <= 2 * OPT (triangle inequality required)
```

Approximation Ratio	Time
2	O(E log V)

6.4 Set Cover Approximation (ln n approximation)

```
GREEDY-SET-COVER(X, F):

U = X (universe of elements)

C = empty (cover)

while U not empty:

S = set in F covering most elements of U

C = C union {S}

U = U - S

return C


Result: |C| <= H(max|S|) * |C*| where H(d) = ln(d) + 1 (harmonic number)
```

Approximation Ratio	Time
O(log n)	O(|X||F|)

6.5 Summary: When to Use What

Problem Type	Algorithm Type	Guaranteed?
Easy (P)	Exact polynomial algorithm	Yes — optimal
NP-Complete (small n)	Exponential exact (backtracking, brute force)	Yes — optimal
NP-Complete (large n, special structure)	DP or greedy	Sometimes
NP-Hard (metric)	Approximation algorithm	Bounded ratio guarantee

NP-Hard (general)	Heuristics / meta-heuristics	No guarantee

Reference: Introduction to Algorithms, 3rd Edition (CLRS) — Cormen, Leiserson, Rivest, Stein